PocketSmart.AI

Complete Project Documentation

AI-powered budget planning & smart product recommendation assistant

Built with FastAPI · Google Gemini AI · Supabase · Cloudinary · Vercel

---

1. Project Overview

PocketSmart.AI is a full-stack web application that uses Google Gemini AI to create personalised budget plans and product recommendations for different life events. You enter a budget and some preferences — the AI does the rest: calculating costs, finding real products, generating a detailed plan, and exporting a clean PDF report.

Core Workflow

```
User enters budget & preferences
        ↓
FastAPI backend validates input
        ↓
Google Gemini AI generates plan + product list
        ↓
Product Service fetches images from Cloudinary
        ↓
Plan saved to Supabase PostgreSQL
        ↓
User sees recommendations page with PDF export
```

---

2. Live Demo & Repository

Resource	Link
Live App (Vercel)	https://pocketsmartai-gamma.vercel.app
GitHub Repository	https://github.com/Pandiyan-Venkatachalam/pocketsmartai
API Health Check	https://pocketsmartai-gamma.vercel.app/api/health
Interactive API Docs	http://127.0.0.1:8080/docs (local only)

---

3. Tech Stack

Layer	Technology
Backend Framework	FastAPI (Python 3.11+)
AI Engine	Google Gemini 1.5 Flash (via google-genai SDK)
Database	PostgreSQL on Supabase (SQLite for local dev)
Image Hosting	Cloudinary (pocketsmart_items folder)
Authentication	JWT tokens (HTTP-only cookies) + bcrypt password hashing
ORM	SQLAlchemy 2.0

Layer	Technology
Frontend	Jinja2 HTML templates + Vanilla CSS + Vanilla JS
ASGI Server	Uvicorn
Deployment	Vercel (Serverless Python)
Testing	Pytest + FastAPI TestClient

---

4. Project Structure

```
PocketSmart/
■■■ api/
■   ■■■ index.py              # Vercel serverless entry point
■■■ app/
■   ■■■ main.py               # FastAPI app factory, middleware, routers
■   ■■■ core/
■   ■   ■■■ config.py         # All settings (reads .env)
■   ■   ■■■ database.py       # SQLAlchemy engine, session, seed data
■   ■   ■■■ security.py       # JWT creation, bcrypt, auth dependencies
■   ■■■ models/
■   ■   ■■■ user.py           # User SQLAlchemy model
■   ■   ■■■ recommendation.py # Recommendation SQLAlchemy model
■   ■■■ schemas/
■   ■   ■■■ auth.py           # Pydantic models for login/register
■   ■   ■■■ home.py           # Home planner input/output schemas
■   ■   ■■■ party.py          # Party planner schemas
■   ■   ■■■ jewelry.py        # Jewelry planner schemas
■   ■   ■■■ custom_planner.py # Custom planner schemas
■   ■   ■■■ recommendation.py # Recommendation output schemas
■   ■■■ routers/
■   ■   ■■■ auth.py           # POST /register, POST /login, GET /me
■   ■   ■■■ home.py           # POST /api/planners/home
■   ■   ■■■ party.py          # POST /api/planners/party
■   ■   ■■■ jewelry.py        # POST /api/planners/jewelry
■   ■   ■■■ custom.py         # POST /api/planners/custom
■   ■   ■■■ dashboard.py      # GET /api/dashboard/stats
■   ■   ■■■ history.py        # GET /api/history, DELETE /api/history/{id}
■   ■   ■■■ web.py            # All HTML page routes (/)
■   ■■■ services/
■   ■   ■■■ gemini_service.py       # Google Gemini AI integration
■   ■   ■■■ home_service.py         # Home planner business logic
■   ■   ■■■ party_service.py        # Party planner business logic
■   ■   ■■■ jewelry_service.py      # Jewelry planner business logic
■   ■   ■■■ custom_service.py       # Custom planner + domain detection
■   ■   ■■■ product_service.py      # Cloudinary product catalog service
■   ■   ■■■ rate_service.py         # Material/labour rate calculations
■   ■   ■■■ recommendation_service.py # Save/fetch recommendation records
■   ■   ■■■ planning/               # Domain-specific planners
■   ■       ■■■ domain_detector.py  # Detects plan type from prompt
■   ■       ■■■ house_planner.py    # House construction logic
■   ■       ■■■ wedding_planner.py  # Wedding budget logic
■   ■       ■■■ travel_planner.py   # Travel budget logic
■   ■       ■■■ business_planner.py # Business budget logic
■   ■       ■■■ school_planner.py   # Education budget logic
■   ■■■ templates/            # Jinja2 HTML templates
■   ■■■ static/
■       ■■■ css/style.css     # Full design system (dark/light mode)
■       ■■■ js/
■           ■■■ main.js       # Sidebar, loading, card interactions
■           ■■■ auth.js       # Login/register/logout JS
■           ■■■ planners.js   # Form submission for all planners
■■■ data/
■   ■■■ mock_products.json    # Fallback product catalog (no Cloudinary)
■■■ scripts/
■   ■■■ setup_supabase.py     # Database migration to Supabase
■   ■■■ backfill_cloudinary.py # Upload images to Cloudinary
■   ■■■ generate_cloudinary_images.py
■■■ tests/
■   ■■■ conftest.py           # Pytest fixtures, test DB
■   ■■■ test_auth.py
■   ■■■ test_planners.py
■   ■■■ test_recommendations.py
■   ■■■ verify_live.py
■■■ .env                      # Your local secrets (never commit)
■■■ .env.example              # Template for environment variables
■■■ requirements.txt          # Python dependencies
■■■ vercel.json               # Vercel routing config
■■■ run.py                    # Local server launcher
```

---

5. Features

For Users

- Home Interior Planner — Budget for furniture, appliances, decor with room type & style preferences
- Party Planner — Wedding/birthday/corporate event with guest count, catering, decor breakdown
- Jewelry Styler — Occasion-based gold/silver/gemstone recommendations
- Custom Planner — Natural language prompt (any goal: house construction, travel, business)
- Dashboard — View all your plans, total savings, utilization charts
- History — Browse, view, and delete past plans
- PDF Export — Clean, printable report with product images and cost breakdown
- Dark / Light Mode — Toggle persists in localStorage
- Secure Auth — Register, login, sessions with JWT + HTTP-only cookies

For Developers

- Mock AI Mode — Works without a Gemini API key (auto-generates realistic plans)
- Auto Fallback — Falls back to mock if Gemini quota is exceeded
- SQLite locally, PostgreSQL in production — Zero config switch via DATABASE_URL
- Interactive API Docs — FastAPI Swagger UI at /docs
- Full Test Suite — Pytest with in-memory SQLite test DB

---

6. Prerequisites

Ensure you have these installed before proceeding:

Tool	Minimum Version	Check
Python	3.11+	`python --version`
pip	23+	`pip --version`
Git	any	`git --version`
Node.js	18+ (for Vercel CLI)	`node --version`

---

7. Local Setup — Step by Step

Step 1 — Clone the Repository

```
git clone https://github.com/Pandiyan-Venkatachalam/pocketsmartai.git
cd pocketsmartai
```

Step 2 — Create a Python Virtual Environment

```
# Windows
python -m venv .venv
.venv\Scripts\activate

# Mac / Linux
python3 -m venv .venv
source .venv/bin/activate
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

Step 4 — Create Your Environment File

```
# Copy the example file
copy .env.example .env        # Windows
cp .env.example .env          # Mac/Linux
```

Then open `.env` and fill in your values (see Section 8).

Step 5 — Run the App

```
python run.py
```

Open your browser: http://127.0.0.1:8080

No Gemini API key? The app still runs in Mock AI mode — it generates realistic, pre-defined plans automatically. You can use every feature without any API key.

---

8. Environment Variables

Create a `.env` file in the project root:

```
# APPLICATION
APP_NAME=PocketSmart AI
APP_ENV=development
DEBUG=True
HOST=127.0.0.1
PORT=8080

# SECURITY / JWT
# Change this to any long random string in production
JWT_SECRET_KEY=change-this-to-a-very-secure-random-string
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=1440    # 24 hours

# DATABASE
# SQLite — works out of the box locally (no server needed)
DATABASE_URL=sqlite:///./pocketsmart.db
# PostgreSQL — for production (Supabase, Railway, etc.)
# DATABASE_URL=postgresql://user:password@host:5432/dbname

# GOOGLE GEMINI AI
# Leave blank to use Mock AI mode
# Get a free key at: https://aistudio.google.com/
GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.6-flash
# auto = use Gemini if key present, else mock
# force_mock = always use mock (for testing)
# live = always use Gemini (fails if no key)
MOCK_AI_MODE=auto

# CLOUDINARY (product images)
# Get free credentials at: https://cloudinary.com/
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret
```

Where to Get API Keys

Service	URL	Free Tier
Google Gemini	https://aistudio.google.com/	Yes (15 req/min)
Cloudinary	https://cloudinary.com/	Yes (25GB storage)
Supabase PostgreSQL	https://supabase.com/	Yes (500MB)

---

9. Running the App Locally

Method 1 — Using run.py (Recommended)

```
python run.py
```

Method 2 — Using uvicorn directly

```
# Development (auto-reload on file changes)
uvicorn app.main:app --host 127.0.0.1 --port 8080 --reload

# Production-style
uvicorn app.main:app --host 0.0.0.0 --port 8080
```

Method 3 — Using the Python module

```
python -m app.main
```

Verify it's running

URL	What You'll See
http://127.0.0.1:8080	Home / Landing page
http://127.0.0.1:8080/login	Login page
http://127.0.0.1:8080/register	Register page
http://127.0.0.1:8080/docs	Swagger UI API Explorer
http://127.0.0.1:8080/api/health	JSON health check

---

10. API Reference

All API endpoints are also available in Swagger UI at `/docs` when running locally.

Authentication

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register a new user	No
POST	/api/auth/login	Login and get JWT token	No
GET	/api/auth/me	Get current user info	Yes
POST	/api/auth/logout	Clear session cookie	Yes

Register example:

```
POST /api/auth/register
{
  "name": "Pandiyan",
  "email": "you@example.com",
```

```
  "password": "SecurePass123!"
}
```

Login example:

```
POST /api/auth/login
{
  "email": "you@example.com",
  "password": "SecurePass123!"
}
```

Response:

```
{
  "access_token": "eyJhbGciOi...",
  "token_type": "bearer",
  "user": { "id": 1, "name": "Pandiyan", "email": "you@example.com" }
}
```

Planners

Method	Endpoint	Description
POST	/api/planners/home	Generate home interior plan
POST	/api/planners/party	Generate party/event plan
POST	/api/planners/jewelry	Generate jewelry recommendations
POST	/api/planners/custom	Custom budget plan (any goal)

Example — Home Planner:

```
POST /api/planners/home
Authorization: Bearer <token>

{
  "total_budget": 150000,
  "room_type": "living_room",
  "style_preference": "modern",
  "room_size_sqft": 250,
  "priority_items": ["sofa", "TV unit", "coffee table"]
}
```

Example — Custom Planner:

```
POST /api/planners/custom
Authorization: Bearer <token>

{
  "prompt": "Plan a 3-day trip to Ooty for 2 people with budget 15000 rupees",
  "budget": 15000
}
```

History and Dashboard

Method	Endpoint	Description
GET	/api/history	List all your past plans
GET	/api/recommendations/{id}	Get a specific plan by ID
DELETE	/api/history/{id}	Delete a plan
GET	/api/dashboard/stats	Dashboard summary stats

Health

Method	Endpoint	Description
GET	/api/health	App health + AI mode status

---

11. Planners Guide

Home Interior Planner

URL: /planners/home

Plan your home furniture, appliances, and decor within a budget.

Inputs:

- Budget (INR) — your total available budget
- Room Type — Living Room, Bedroom, Kitchen, Bathroom, Entire Flat
- Style — Modern, Traditional, Minimalist, Bohemian, Scandinavian
- Room Size — in square feet
- Priority Items — specific items you want (e.g., sofa, refrigerator)

AI Output: Budget feasibility, itemized product recommendations with prices, cost breakdown by category, risks and alternatives.

Party Planner

URL: /planners/party

Plan any event — birthday, wedding, corporate — with full budget allocation.

Inputs:

- Budget (INR)
- Event Type — Birthday, Wedding, Anniversary, Corporate, Baby Shower
- Guest Count
- Venue — Home, Hall, Outdoor, Restaurant
- Priorities — Food, Decor, Photography, Entertainment

AI Output: Per-head cost calculation, category-wise budget allocation, vendor recommendations, savings tips.

Jewelry Styler

URL: /planners/jewelry

Find the right jewelry pieces for your occasion within budget.

Inputs:

- Budget (INR)
- Occasion — Wedding, Festival, Daily Wear, Gift, Casual
- Metal Preference — Gold, Silver, Platinum, Artificial
- Style — Traditional, Modern, Indo-Western

AI Output: Jewelry set recommendations with images, live gold/silver rate estimates, mix-and-match suggestions.

Custom Planner

URL: /planners/custom

Type any budget goal in plain English. The AI detects the domain automatically.

Examples you can type:

```
"Build a 2BHK house in Tamil Nadu for 25 lakhs"
"Plan my daughter's wedding for 5 lakhs"
"Start a small tea shop business for 50,000 rupees"
"Plan a 7-day Europe trip for 2 people, budget 3 lakhs"
"Setup a school computer lab for 2 lakh budget"
```

Detected domains: House Construction, Wedding, Travel, Business, School/Education, General Shopping

---

12. AI Modes — Live Gemini vs Mock

Mode	When Active	Quality
Live Gemini	GEMINI_API_KEY is set and valid	Highly personalised, real AI plans
Mock AI	No API key or quota exceeded	Pre-generated realistic plans
Force Mock	MOCK_AI_MODE=force_mock	Always mock (for testing)

Check which mode is active:

```
curl http://127.0.0.1:8080/api/health
# Response: { "gemini_live": true/false, "mode": "live_gemini" / "mock_ai" }
```

Mock AI is great for demos and development. The plans look realistic and all UI features work exactly the same.

---

13. Database

Local Development (SQLite)

SQLite is used by default — no setup needed. A `pocketsmart.db` file is created automatically.

```
DATABASE_URL=sqlite:///./pocketsmart.db
```

Production (PostgreSQL via Supabase)

```
DATABASE_URL=postgresql://user:password@host:5432/dbname
```

Schema

users table:

```
id              INTEGER PRIMARY KEY
name            VARCHAR(100)
email           VARCHAR(255) UNIQUE
password_hash   VARCHAR(255)
created_at      DATETIME
```

recommendations table:

```
id                  INTEGER PRIMARY KEY
user_id             INTEGER (FK → users.id)
planner_type        VARCHAR(20)    # home / party / jewelry / custom
budget              FLOAT
total_estimated_cost FLOAT
remaining_budget    FLOAT
input_data          JSON           # original user inputs
plan_data           JSON           # AI-generated plan
ai_notes            TEXT
```

```
is_mock_ai          BOOLEAN
created_at          DATETIME
```

Seed Data

On first startup, a demo user is created:

- **Email:** alex.demo@pocketsmart.ai
- **Password:** PocketSmart2026!

---

14. Cloud Services

Cloudinary (Product Images)

- Cloud Name: `dqr1zbyeb`
- Folder: `pocketsmart_items`
- If unavailable, mock product images from Unsplash are used as fallback

Supabase (Production Database)

- Region: AP Northeast 1 (Tokyo)
- Connection: IPv4 Pooler (Vercel serverless compatible)

Vercel (Hosting)

- Live URL: https://pocketsmartai-gamma.vercel.app
- Runtime: Python 3.12 Serverless
- Entry: api/index.py imports FastAPI app

---

15. Running Tests

```
# Run all tests
pytest

# Verbose output
pytest -v

# Run specific test file
pytest tests/test_auth.py -v
pytest tests/test_planners.py -v
pytest tests/test_recommendations.py -v

# With coverage report
pytest --cov=app tests/
```

Tests use in-memory SQLite — completely isolated from development data.

Test Files

File	What It Tests
test_auth.py	Register, login, duplicate email, bad password
test_planners.py	Home/party/jewelry/custom plan generation
test_recommendations.py	Save, fetch, delete recommendation history
test_domains.py	Domain auto-detection for custom planner

File	What It Tests
verify_live.py	Live endpoint smoke test (run against production)

---

16. Deploying to Vercel

First-Time Setup

Step 1 — Install Vercel CLI

```
npm install -g vercel
```

Step 2 — Login

```
vercel login
```

Step 3 — Deploy

```
cd PocketSmart
npx vercel --prod --yes
```

Step 4 — Add Environment Variables

Go to https://vercel.com → Your Project → Settings → Environment Variables:

Key	Value
DATABASE_URL	Your Supabase PostgreSQL connection string
GEMINI_API_KEY	Your Google Gemini API key
JWT_SECRET_KEY	A long random secret string
CLOUDINARY_CLOUD_NAME	Your Cloudinary cloud name
CLOUDINARY_API_KEY	Your Cloudinary API key
CLOUDINARY_API_SECRET	Your Cloudinary API secret
APP_ENV	production
DEBUG	False
MOCK_AI_MODE	auto

Re-Deploying After Code Changes

```
# Option 1: Push to GitHub
git add .
git commit -m "your changes"
git push origin main

# Option 2: Deploy directly
npx vercel --prod --yes
```

Key Vercel Files

```
// vercel.json
{
  "routes": [
    { "src": "/(.*)", "dest": "api/index.py" }
  ]
}

# api/index.py
from app.main import app
```

---

17. Demo Credentials

Role	Email	Password
Demo User	alex.demo@pocketsmart.ai	PocketSmart2026!

Security Note: Change JWT_SECRET_KEY to a strong random value before sharing the app publicly.

---

18. Troubleshooting

App won't start — ModuleNotFoundError

```
# Activate virtual environment
.venv\Scripts\activate       # Windows
source .venv/bin/activate    # Mac/Linux

# Reinstall
pip install -r requirements.txt
```

Database errors on first run

```
# Delete old database and let it recreate
del pocketsmart.db           # Windows
rm pocketsmart.db            # Mac/Linux
python run.py
```

GEMINI_API_KEY not set warning

This is normal — app runs in Mock AI mode. To enable live AI:

1. Get free key at https://aistudio.google.com/

2. Add `GEMINI_API_KEY=your-key` to `.env`

3. Restart the server

Vercel deploy fails — "Project names can be up to 100 characters"

```
npx vercel --prod --yes
# When prompted for project name, enter: pocketsmartai
```

PDF prints dark background

- Use the Print PDF button or Ctrl+P
- In browser print dialog, enable "Background graphics" for color images

Light mode not applying to some cards

- Hard refresh: Ctrl+Shift+R (clears CSS cache)

Supabase Connection Refused

- Supabase pauses free-tier databases after inactivity
- Go to https://supabase.com → your project → click "Restore project"

Quick Reference

```
# Clone and setup
git clone https://github.com/Pandiyan-Venkatachalam/pocketsmartai.git
cd pocketsmartai
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env        # then edit .env
python run.py                 # → open http://127.0.0.1:8080

# Run tests
pytest -v

# Deploy to Vercel
git add . && git commit -m "update" && git push origin main
npx vercel --prod --yes

# Check health
curl http://127.0.0.1:8080/api/health
```

PocketSmart.AI — Built with FastAPI + Google Gemini AI · Deployed on Vercel · Data on Supabase